Лабораторная работа №3
Шаблоны и Исключения

Цель работы:

Получение практических навыков разработки и отладки программ с использованием шаблонов классов и механизма обработки исключений.

ПОРЯДОК ВЫПОЛНЕНИЯ РАБОТЫ

1. Ознакомиться с общей постановкой задачи.
2. Ознакомится с вариантом задания – соответствует вашему номеру в списке группы.
3. Разработать классы согласно варианту задания.
4. Реализовать механизм обработки исключений для обеспечения надежности работы программы.
5. Написать и отладить программу на подготовленных наборах тестовых данных.
6. Подготовить отчет по лабораторной работе. Отчет должен включать в себя:
   - титульный лист;
   - цель лабораторной работы;
   - постановку задачи;
   - текст программы с комментариями;
   - демонстрацию работы программы *(Снимки экрана при выполнении действий программы с описанием).*
   - краткие ответы на контрольные вопросы;
   - выводы по лабораторной работе.
7. Защитить лабораторную работу перед преподавателем.

КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Какой принцип ООП реализуется с помощью шаблонных функций и классов?
2. Когда целесообразно использовать шаблоны функций?
3. Каков общий вид объявления шаблона функции?
4. Каков синтаксис объявления шаблонного класса?
5. В чем особенности определения шаблонного класса?
6. В чем особенности определения метода шаблонного класса вне класса?
7. Что такое специализация шаблона?
8. Как объявить объект шаблонного класса?
9. Что такое инстанцирование?
10. Что такое исключительная ситуация?
11. Для чего предназначены блоки try, catch, throw?
12. Как работает раскрутка стека?
13. В какой последовательности программа обрабатывает исключительную ситуацию?
14. Что произойдет, если исключение окажется не перехваченным?
15. Как создать собственный класс исключения?
16. В чем преимущество использования исключений перед возвратом кодов ошибок?
17. Чем оператор throw отличается от оператора return?
18. Что такое перехват исключений по ссылке и почему это важно?
19. В каком месте среди блоков catch должен располагаться блок catch(…)?
20. Что означает идентификатор b в следующей строке с catch-блоком? catch (int b)

ОБЩАЯ ПОСТАНОВКА ЗАДАЧИ

Необходимо разработать программу, в которой будет реализован шаблонный класс, выполняющий обработку массива данных согласно варианту задания. **Особое внимание уделить обработке исключительных ситуаций.**

Общее задание для всех вариантов (Обязательная часть, связанная с исключениями):

1. **Конструкторы и деструктор:** Реализовать конструктор по умолчанию, конструктор копирования, конструктор перемещения (move constructor), оператор копирующего и перемещающего присваивания. Обеспечить строгую гарантию исключений (strong exception safety) для всех методов, изменяющих состояние объекта.
2. **Безопасность методов:** Все методы, работающие с динамической памятью или выполняющие критичные операции, должны обеспечивать как минимум базовую гарантию исключений (no resource leaks).
3. **Проверка входных данных:** Методы, получающие параметры извне (например, индексы), должны проверять их корректность (выход за границы массива, нулевые указатели и т.д.) и генерировать соответствующие исключения (например, std::out_of_range, std::invalid_argument).
4. **Обработка исключений в main:** Продемонстрировать обработку исключений в функции main() с помощью try-catch блоков, обеспечивая корректное завершение программы или восстановление после ошибок.

Дополнительные задания (в зависимости от оценки):

Оценка «Удовлетворительно» (Базовый уровень)

- **Что должен сделать студент:**
  1. Выполнить обязательную часть.
  2. Написать методы ввода/вывода данных, принимающие потоки в качестве параметров.
  3. Реализовать один метод из варианта.

Оценка «Хорошо» (Продвинутый уровень)

- **Что должен сделать студент:**
  1. **Доработать программу с оценки «Удовлетворительно».**
  2. Перегрузить операции ввода/вывода (>>, <<), вызвав в них методы ввода/вывода.
  3. Выполнить второй метод из варианта.

Оценка «Отлично» (Углубленный уровень)

- **Что должен сделать студент:**
  1. **Доработать программу с оценки «Хорошо».**
  2. Реализовать собственный класс исключений. Создать хотя бы один собственный класс исключения, унаследованный от std::exception, для обработки специфичных для вашей предметной области ошибок (например, FigureValidationError, MatrixDimensionError).
  3. Реализовать дополнительные методы (например, GenerateRandomValues(), модификация элемента, перегрузка операторов сравнения).

ВАРИАНТЫ ЗАДАНИЙ

Вариант №1.

Напишите программу, в которой создайте шаблонный класс **TSimpleMatrix**, реализующий динамический двумерный массив и функции-методы:

- определения максимального значения и его индекса среди строк, в которых больше половины элементов имеют не отрицательные значения;
- вычисления суммы абсолютных значений отрицательных элементов.

Протестируйте вашу программу для работы с массивами различных типов: **int**, **double** и класса **дробное число** (нужно реализовать с перегрузкой необходимых операторов и функций).

Вариант №2.

Напишите программу, в которой создайте шаблонный класс **TVectorArray**, реализующий динамический одномерный массив и функции-методы:

- сортировки массива;
- подсчёта количества векторов (и вывод их на экран), длина которых больше средней длины векторов массива.

Протестируйте вашу программу для работы с массивами различных типов: **T2DVector** и **T3DVector**. Эти типы необходимо создать (они представляют собой вектора, обладающие начальной и конечной точкой в двухмерном (x,y) и трехмерном (x,y,z) пространстве, где координаты представлены вещественными числами). Обработка на основе длин векторов.

Вариант №3.

Напишите программу, в которой создайте шаблонный класс **TPointMatrix**, реализующий динамический двумерный массив и функции-методы:

- определения точки, максимально удаленной от остальных элементов в своей строке;
- поиска точек, находящихся в заданной четверти декартового пространства (четверть указывает пользователь).

Протестируйте вашу программу для работы с массивами для типов **Двумерная точка** и **Трехмерная точка**. Эти типы необходимо создать.

Вариант №4.

Напишите программу, в которой создайте шаблонный класс **TSquareMatrix**, реализующий динамический двумерный массив (с равными сторонами) и функции-методы:

- обмена значений строки с максимальным элементом на столбец с минимальным элементом;
- подсчёта суммы элементов для строк.

Протестируйте вашу программу для работы с массивами различных типов: **double**, **дробное число** (которое представлено двумя компонентами). Этот класс необходимо создать.

Вариант №5.

Напишите программу, в которой создайте шаблонный класс **TMatrix**, реализующий динамический двумерный массив и функции-методы:

- подсчета элементов массива (с отображением на экран), которые являются положительными и меньше среднеарифметического значения всех элементов своей строки;
- расчёта суммы отрицательных элементов массива.

Протестируйте вашу программу для работы с массивами различных типов: **float,** класса **дробное число** (которое представлено двумя компонентами). Этот класс необходимо создать.

Вариант №6.

Напишите программу, в которой создайте шаблонный класс **TFigureArray**, реализующий динамический одномерный массив и функции-методы:

- сортировки массива по площади фигуры (любой метод);
- поиска элементов, у которых периметр больше среднеарифметического значения по всем элементам.

Протестируйте вашу программу для работы с массивами различных типов: **Квадрат** (описан точками с вещественными координатами (x,y)), **Окружность** (задана точка центра (x,y) и радиус). Эти типы необходимо создать.

Вариант №7.

Напишите программу, в которой создайте шаблонный класс **TMatrix**, реализующий динамический двумерный массив и функции-методы:

- сортировки выбранной строчки массива методом Шелла (по выбору пользователя);
- поиска максимального и минимального элементов в строках.

Протестируйте вашу программу для работы с массивами различных типов: **int,** класса **дробное число** (представлено двумя компонентами). Этот класс необходимо создать.

Вариант №8.

Напишите программу, в которой создайте шаблонный класс **TFigureArray**, реализующий динамический одномерный массив и функции-методы:

- поиска элемента, у которых одна из точек больше всего удалена от начала координат, т.е. точки (0,0,0);
- Расчета суммарного периметра.

Протестируйте вашу программу для работы с массивами различных типов: **Четырехугольник, Шестиугольник** (эти типы представляют собой фигуры описанные точками (x,y,z) где координаты представлены вещественными числами). Эти типы необходимо создать.

Вариант №9.

Напишите программу, в которой создайте шаблонный класс **TMatrix**, реализующий динамический двумерный массив (с не равными сторонами) и функции-методы:

- Расчёта суммы положительных элементов массива для четных строк;
- Нахождения среднеарифметического значения для элементов между минимальным и максимальным элементами массива в каждом столбце.

Протестируйте вашу программу для работы с массивами различных типов: **double**, **к**ласса **дробное число** (которое представлено двумя компонентами). Этот класс необходимо создать.

Вариант №10

Напишите программу, в которой создайте шаблонный класс **TFigureArray**, реализующий динамический одномерный массив и функции-методы:

- сортировка массива на основе периметра фигуры (любой метод);
- поиска элементов с минимальным и максимальным периметром.

Протестируйте вашу программу для работы с массивами различных типов: **Треугольник, Пятиугольник** (эти типы представляют собой фигуры описанные точками (x,y,z) где координаты представлены вещественными числами). Эти классы необходимо создать.

Вариант №11.

Создайте шаблонный класс **TComplexVector** для работы с массивом комплексных чисел включающий и функции-методы:

- вычисления суммы комплексных чисел по массиву;
- поиска минимального элемента.

Реализуйте шаблонный класс комплексного числа **TComplex**, задавая вещественную и мнимую части как числами типа **int**, так и **дробными числами** (которые представлены двумя компонентами). Этот класс необходимо создать.

Вариант №12.

Напишите программу, в которой создайте шаблонный класс **TFigureMatrix**, реализующий динамический двумерный массив и функции-методы:

- сортировки строчек массива (метод Шелла) по средней площади фигуры, средняя площадь фигур рассчитывается для каждой строки;
- поиска элементов, у которых периметр больше среднеарифметического значения для нечетных строк.

Протестируйте вашу программу для работы с массивами различных типов: **Треугольник, Прямоугольник** (эти типы представляют собой фигуры описанные точками (x,y,z) где координаты представлены вещественными числами). Эти классы необходимо создать.

Вариант №13.

Напишите программу, в которой создайте шаблонный класс **TFigureArray**, реализующий динамический одномерный массив и функции-методы:

- сортировка массива по площади фигуры (метод Шелла);
- поиск элементов, у которых площадь больше среднеарифметического значения.

Протестируйте вашу программу для работы с массивами различных типов: **Квадрат, Круг** (эти типы представляют собой фигуры описанные точками (x,y), где координаты представлены вещественными числами). Эти классы необходимо создать.

Вариант №14.

Напишите программу, в которой создайте шаблонный класс **TMatrix**, реализующий динамический двумерный массив и функции-методы:

- вычисления среднеарифметического значения, среди элементов массива расположенных на побочной диагонали;
- сортировки выбранной строки.

Протестируйте вашу программу для работы с массивами различных типов: **float** и класса **дробное число** (которое представлено двумя компонентами). Этот класс необходимо создать.

Вариант №15.

Напишите программу, в которой создайте шаблонный класс **TMatrix**, реализующий динамический двумерный массив и функции-методы:

- вычисления суммы элементов, которые меньше среднеарифметического значения элементов левой половины массива (все элементы, номер столбца которых < M/2);
- нахождения наибольшего значения среди минимальных значений по каждой строке массива.

Протестируйте вашу программу для работы с массивами различных типов: **double** и класса **дробное число** (которое представлено двумя компонентами). Этот класс необходимо создать.

Вариант №16.

Напишите программу, в которой создайте шаблонный класс **TFigureMatrix**, реализующий динамический двумерный массив (с не равными сторонами) и функции-методы:

- поиска наименьшего значение среди элементов массива с выводом его индекса и значения на экран;
- определения количества элементов массива, которые меньше среднего арифметического значения всех элементов своего столбца с указанием в каком столбце сколько элементов.

Протестируйте вашу программу для работы с массивами различных типов: **Треугольник, Пятиугольник** (эти типы представляют собой фигуры описанные точками (x,y) где координаты представлены вещественными числами). Вся обработка ведется на основе значения периметра фигуры. Эти классы необходимо создать.

Вариант №17.

Напишите программу, в которой создайте шаблонный класс **TMatrix**, реализующий динамический двумерный массив (с не равными сторонами) и функции-методы:

- Нахождения номера строки, в которой максимальное значение является минимальным среди аналогичных значений других строк;
- Определения количества элементов массива, которые больше среднего арифметического значения элементов нечетных строк массива.

Протестируйте вашу программу для работы с массивами различных типов: **int** и класса **дробное число** (которое представлено двумя компонентами). Этот класс необходимо создать.

Вариант №18.

Напишите программу, в которой создайте шаблонный класс **TMatrix**, реализующий динамический двумерный массив (с не равными сторонами) и функции-методы:

- определение максимального элемента и его индексов, среди элементов массива главной диагонали;
- вычисления суммы элементов в строке, у которой среднее значение её элементов максимально.

Протестируйте вашу программу для работы с массивами различных типов: **double** и класса **дробное число** (которое представлено двумя компонентами). Этот класс необходимо создать.

Вариант №19.

Напишите программу, в которой создайте шаблонный класс **TPointsArrays**, реализующий два динамических одномерных массива (множества) и функции-методы:

- определения множества, в котором больше всего точек имеют значения больше заданного (указывает пользователь);
- вычисления количества точек, которые находятся на расстоянии больше среднего значения расстояний для массивов (расстояние рассчитывать только между элементами массивов, обладающими одинаковыми индексами).

Протестируйте вашу программу для работы с массивами различных типов: **int** и класс **T3DPoint** (описывающий точки в трехмерном (x,y,z) пространстве, где координаты представлены вещественными числами). Этот класс необходимо создать.

Вариант №20.

Напишите программу, в которой создайте шаблонный класс **TMatrix**, реализующий динамический двумерный массив и функции-методы:

- кругового сдвига выбранной пользователем строчки;
- подсчёта количества элементов, которые являются минимальными среди своего окружения (т.е. в строке и столбце с индексами анализируемого элемента).

Протестируйте вашу программу для работы с массивами различных типов: **int**, **char**, **float, double**.

Вариант №21.

Напишите программу, в которой создайте шаблонный класс **TMatrix**, реализующий динамический двумерный массив и функции-методы:

- определения количества элементов массива, которые меньше среднего арифметического значения всех элементов четных строк;
- нахождения суммы положительных элементов массива (для комплексного числа проверяется действительная часть).

Протестируйте вашу программу для работы с массивами различных типов: **double** и класса **комплексное число** (представлено двумя компонентами). Этот класс необходимо создать.

Вариант №22.

Напишите программу, в которой создайте шаблонный класс **TPointMatrix**, реализующий двумерный массив и функции-методы:

- поиска элемента наиболее удаленного от среднеарифметического значения по всем элементам массива;
- подсчета сумм положительных и отрицательных элементов массива для нечетных столбцов, при этом элемент считать отрицательным, если одна из координат точки меньше нуля.

Протестируйте вашу программу для работы с массивами различных типов: **T2DPoint** и **T3DPoint** (описывающие точки в двухмерном (x,y) и трехмерном (x,y,z) пространствах – координаты представлены вещественными числами). Эти классы необходимо создать.

Вариант №23

Напишите программу, в которой создайте шаблонный класс **TPointsArrays**, реализующий три и более динамических одномерных массива (множества) и функции-методы:

- определения множества, у которого больше всего точек находятся на расстоянии <=**X** от среднеарифметического значения (значение **X** указывает пользователь);
- определения множеств, у которых две точки с одинаковыми индексами находятся на максимальном расстоянии (расстояние рассчитывать только между элементами массивов, обладающими одинаковыми индексами и если один массив больше другого, то обработка ведется по наименьшему количеству элементов).

Протестируйте вашу программу для работы с массивами различных типов: **int** и класс **T3DPoint** (описывающий точки в трехмерном (x,y,z) пространстве, где координаты представлены вещественными числами). Этот класс необходимо создать.

Вариант №24.

Напишите программу, в которой создайте шаблонный класс **TMatrix**, реализующий динамический двумерный массив и функции-методы:

- вычисления суммы элементов главной диагонали;
- нахождения наименьшего значения среди элементов массива, больших среднеарифметического значения по всему массиву.

Протестируйте вашу программу для работы с массивами различных типов: **int** и класса **дробное число** (которое представлено двумя компонентами). Этот класс необходимо создать.

Вариант №25.

Напишите программу, в которой создайте шаблонный класс **TMatrix**, реализующий динамический двумерный массив и функции-методы:

- вычисления суммы элементов в левой половине матрицы (столбцы < M/2), которые меньше среднего арифметического этих элементов;
- нахождения наибольшего значения среди минимальных элементов каждой строки.

Протестируйте вашу программу для работы с массивами различных типов: **double**, **Дробь**.

Вариант №26.

Напишите программу, в которой создайте шаблонный класс **TMatrix**, реализующий динамический двумерный массив и функции-методы:

- рекурсивного поиска седловой точки (элемент, минимальный в своей строке и максимальный в своем столбце);
- транспонирования матрицы.

Протестируйте вашу программу для работы с массивами различных типов: **float**, **double**.

Вариант №27.

Напишите программу, в которой создайте шаблонный класс **TFigureArray**, реализующий динамический одномерный массив и функции-методы:

- реализации итераторов для прямого и обратного обхода массива;
- поиска первой фигуры, у которой площадь больше заданного значения, с использованием итераторов.

Протестируйте вашу программу для работы с массивами различных типов: **Квадрат**, **Окружность**.

Вариант №28.
Напишите программу, в которой создайте шаблонный класс **TMatrix**, реализующий динамический двумерный массив и функции-методы:
▪ перемножения двух матриц;
▪ вычисления определителя матрицы (для квадратных матриц).
Протестируйте вашу программу для работы с массивами различных типов: **int**, **double**.

Вариант №29.
Напишите программу, в которой создайте шаблонный класс **TSparseMatrix** для работы с разреженными матрицами (большинство элементов нули) и функции-методы:
▪ эффективного хранения матрицы;
▪ сложения двух разреженных матриц.
Протестируйте вашу программу для работы с типами: **int**, **double**.

Вариант №30.
Напишите программу, в которой создайте шаблонный класс **TPriorityFigureArray**, реализующий массив с приоритетной очередью (на основе кучи) и функции-методы:
▪ вставки новой фигуры в очередь с сохранением свойства кучи;
▪ извлечения фигуры с максимальным периметром (или площадью).
Протестируйте вашу программу для работы с массивами различных типов:
Треугольник, **Прямоугольник**.